Assignment No 2

Q1]

Ans]

```
#include<stdio.h>
# define N,s

Struct Student{
   int id,source;
   char name[20];
} S[N], t;

int main(){
   int i,j,ch,id, found;

  for(i=0 ; i<N; i++ ){
   printf(" Enter Id, Name, Score: ");
   scanf("%d %19s %d", &s[i].id, s[i].name, &s[i].source);
}
printf("\n 1. Bubble sort  2. selection sort ");
printf("\n 3. linear search  4. Binary search \n choice ");
 scanf("%d", &ch);

if(ch==1 || ch==2){
  For(i=0; i<N-1; i++){
    For(j=i+1; j< N; j++ ){
    if((ch==1 && s[i].source < s[j].source) ||
        (ch==2 && s[i].source < s[j].source)){
         t=s[i]; s[i] = s[j] ; s[j]=t;
  }}}
For (i=0; i<N; i++){
printf("%d %s %d Rank: %d \n",
   s[i].id, s[i].name, s[i].score, i+1);
}
```

```
else{
   printf("Enter ID to Search:");
   scanf("%d",&id);
   found=0;

 if(ch==3){
 for(i=0;i<N;i++){
   if(s[i].id==id){
   printf("found:%s sore :%d\n",s[i].name,
      s[i].sore);
         found=1; break;
 }}
 else if(ch==4){
  for(i=0;i<=N-1;i++)
     for(j=i+1;j<N;j++)
       if(s[i].id>s[j].id){
         t=s[i]; s[i]=s[j]; s[j]=t;
  }
  i=0; j=N-1;
  while(i<=j){
   int m=(i+j)/2;
   if(s[m].id==id){
   printf("found:%s source:%d \n",
      s[m].name, found=1; break;
   }
  if(s[m].id<id) i=m+1;
   else j=m-1;
 }
 if(!found) printf("student not found!\n");
 }
 return 0;
 }
```

Q2

Ans

Dataset size	Bubble Sort	Selection Sort	Linear search	Binary search
10	45	45	10	4
100	4,950	4,950	100	7
500	124,750	124,750	500	9

Q3

Ans] Most suitable sorting and searching Algorithm

- Bubble Sort: Simple but inefficient for large datasets.
  Time complexity : $O(n^2)$

- Selection Sort: Simple and performs a predictable number of comparisons.
  Time complexity: $O(n^2)$

- Linear ~~Sort~~ search: Suitable for small or unsorted dataset.
  Time complexity: $O(n)$

- Binary Search: Suitable for sorted records and efficient retrieval.
  Time complexity: $o(\log n)$